



本格的なアプリケーション化を行ないたいです。
まずはそれについてどう思いますか？
まずはPCで、管理者権限でログインのみでよいと思っています。
ロゴ等含め全て他の製品と同じクオリティでできれば思っているのですが、本格的な手順を詳細に教えてください。
これをClaudeに渡そうと思います。



結論から言うと、いまの仕様を「本物のアプリ」に落とし込むタイミングとしてちょうど良いです。
PC・管理者だけログインの単一ユーザ構成で始めるのも妥当ですが、最初から将来の多ユーザ／本番運用を見越した骨格だけは入れておいた方がいいです。
以下、そのまま Claude に渡せるように「21章：本格アプリケーション化ロードマップ」として書きます。
(既存仕様の後ろにコピペで追記する想定です)

21. 本格アプリケーション化ロードマップ (PC・管理者ログイン版)

21.0 3行テイクアウェイ

- まず 1 本の“縦スライス” (ログイン → データアップロード → 契約チェック → 因果推定 → DecisionCard 表示) を本番クオリティで完成させる。
- 認証は当面 **「単一管理者アカウント」** に絞るが、DB スキーマと API は将来の多ユーザ／ロール拡張を前提に設計する。
- 配布形態は **Docker Compose 1 コマンド** で **PC ローカル完結** を目標とし、ロゴ／テーマ／文言まで含めて「他製品と同等の SaaS UI 品質」を目指す。

21.1 前提とスコープ (アプリケーション化 v1)

- 対象環境
 - PC (Linux/Fedora を前提。将来 Windows/WSL, macOS にも拡張可能な構成)。
 - `localhost:3001` 等でブラウザアクセスする Web アプリ。
- ユーザ／権限
 - v1 は **管理者 1 アカウントのみ**。
 - ただし DB には `users` テーブル + `roles` カラムを用意し、将来 `analyst/viewer` を追加可能にする。
- データ
 - 本番 CSV/Parquet を **手動アップロード** (DWH 直結は v2 以降)。
 - データはローカルの `artifacts/` (Parquet) + `postgres` に保存。
- 非スコープ (v1 ではやらない)
 - マルチテナント認証 (IdP 連携、SSO)。
 - モバイル最適化。
 - リアルタイム配信連携 (Kafka/Flink)。

21.2 実装フェーズ概要

Phase 0: リポジトリ整備と環境分離

- ディレクトリ構成の確定
 - 仕様 13 章の構成をリポジトリに反映し、`backend/`, `frontend/`, `config/`, `infra/` を作成。
- 設定ファイルの分離
 - `config/app.env.example`
 - `APP_ENV=local|staging|prod`
 - `DATABASE_URL=postgresql://cqox:...`
 - `SECRET_KEY=...` など。
 - `.env` は gitignore。
- Docker Compose 雛形**
 - `infra/docker-compose.local.yml`
 - サービス: `backend`, `frontend`, `postgres`, `prometheus`, `grafana` (必要最低限)。

4. CI の最小セット

- github/workflows/ci.yml に lint + pytest だけでもよいので追加。

Phase 1: 認証・権限の骨格 (単一管理者版)

1. DB スキーマ

- users テーブル (PostgreSQL)
 - id, email, password_hash, role, created_at, updated_at
- v1 では role='admin' の 1 行だけを手動で投入。

2. 認証 API

- POST /api/auth/login
 - 入力: email, password
 - 成功時: 短期 JWT もしくはサーバセッション ID を返却。
- POST /api/auth/logout
- GET /api/auth/me

3. FastAPI の保護ルート

- Depends(get_current_user) で /api/* を保護。
- v1 は admin のみ許可だが、将来 role in {admin, analyst} に拡張できる形で実装。

4. フロントエンド

- /login ページを新設、成功時にトークンを localStorage か cookie に保存。
- 既存のアプリケーションページは、未ログイン時 /login ヘリダイレクト。

Phase 2: 「縦スライス」完成 (エンドツーエンド)

1. バックエンド

- 既存仕様の
POST /api/upload → POST /api/validate/contract → POST /api/analyze/comprehensive → GET /api/results/{job_id}
を「admin 認証前提」で確実に動作させる。
- Job キューは v1 ではシンプルに in-process か DB ベースで OK (Celery 等は v2 以降)。

2. フロントエンド

- 左メニュー
 - データ管理 / Policy Lab / Causal Design / Diagnostics / Portfolio & ROI / Digital Twin / Export Gate / Decision Console
- v1 の最重要画面
 - Causal Design & Evaluation**
 - データセット選択 → カラムマッピング確認 → Estimator 選択 → 「Train Models」 → 進捗バー → Decision Card。
 - Decision Console**
 - 主要 KPI (Total Incremental Profit / Average Δ¥ / Total Decisions) と Decision Cards 一覧。
- UI 上は既存モックのスクリーンショットと同等の見た目を実現する (色・フォント・余白まで合わせる)。

3. テスト

- pytest で
 - 認証がないと 401 になること。
 - 正しい CSV を投げると 200 で Job が完了し、結果 JSON に Δ¥ が含まれること。
- Playwright/E2E
 - login → upload → analyze → DecisionCard 表示 を 1 本書く。

Phase 3: ログ/ブランド/UI デザインの仕上げ

1. ブランド設計

- キーワード例: 「Causal」「Query」「Optimizer」「夜明け」「深い青」。
- カラーパレット (例)
 - プライマリ: #1B2745 (ネイビーブルー)
 - アクセント: #FF5C7A (キャンペーン強調色)
 - サクセス: #22C55E, ワーニング: #F97316, エラー: #EF4444
- タイポグラフィ
 - 見出し: Inter / Noto Sans JP
 - 本文: Noto Sans JP

2. ロゴ制作

- 要件
 - 単色・ライト／ダーク背景両対応。
 - cqox ロゴタイプ+シンプルな因果グラフアイコンを組み合わせた SVG。
 - サイズ: 16×16, 32×32, 128×128, 512×512。
- 納品物
 - frontend/public/logo_cqox.svg
 - frontend/public/favicon.ico
 - frontend/public/site.webmanifest

3. UI デザインシステム

- Figma (または代替) でコンポーネントライブラリを作成 (Button, Card, Tabs, Badge, Tooltip, Banner)。
- React 側は shadcn/ui ベースでコンポーネントを実装し、テーマを上記パレットに統一。

4. コピー／文言の整備

- すべてのページで
 - 英語と日本語のラベルを統一 (例: 「Causal Design & Evaluation / 因果設計と評価」)。
 - ツールチップに用語定義 (ATE, CATE, Overlap, CAS Score など) を短く記載。

Phase 4: 配布と運用 (PC ローカル向け)

1. Docker Compose 一発起動

- コマンド:

```
docker compose -f infra/docker-compose.local.yml up -d
```

- 起動後、ユーザは `http://localhost:3001` にアクセスし、管理者アカウントでログイン。

2. 初期管理者アカウントの設定

- `.env` で `ADMIN_EMAIL`, `ADMIN_PASSWORD` を設定。
- 初回起動時にバックエンドが `users` テーブルを確認し、存在しなければハッシュ化して登録。

3. バックアップ

- `postgres` ボリュームと `artifacts/` ディレクトリを `tar` で固めてバックアップ。

4. ログ／メトリクス

- ログ: `logs/cqox_backend.jsonl` (構造化 JSON)。
- Prometheus/Grafana に基本メトリクス
 - HTTP レイテンシ
 - Job 所要時間
 - 契約エラー (HTTP 400) の件数

21.3 具体タスク一覧 (Claude への指示テンプレ)

Claude に渡す時は、例えば次のようなタスク列として指示できる。

1. Auth 実装

- 仕様 21.2 Phase1 に従い、`users` テーブルと FastAPI + React のログイン機能を実装。

2. 縦スライス

- ログイン済み状態で、`upload → contract → analyze → results` が一気通貫で動くようにバックエンドとフロントエンドを接続。

3. デザイン適用

- 仕様 21.3 のカラーパレットとロゴをもとに、全ページのヘッダー／サイドナビ／カードコンポーネントを統一。

4. Docker 化

- `infra/docker-compose.local.yml` を作成し、`docker compose up` でローカル完結するようにする。

5. E2E テスト

- Playwright で「ログイン→アップロード→分析→DecisionCard 表示」シナリオの自動テストを書く。

What this does (この章の役割)

- ゴール: モック UI から「PC 上で動く、本物のアプリケーション」に昇格させるための、実装順序と必要タスクを明確にする。
- 入力: 既存の CQOX 仕様 (章 0~20) と、ローカル PC で動かしたいという要件。
- アウトプット:

- 認証・UI・ロゴ・Docker 化・テストまでの **具体フェーズ**
- Claude や他モデルに渡しても迷わないレベルの **実装 TODO** リスト。

Expert insight (Google/Meta/NASA レベル)

■ **本番クオリティにする時は「横に広げる」前に必ず 1 本の“縦スライス”を完成させる。**

ログイン → アップロード → 契約チェック → 推定 → 可視化 → ログ・メトリクスまでを

1 パスで完走する経路を先に固めると、その後の機能追加は「そのパスの横に枝を増やす」作業になる。

大規模プロダクトでもこの“縦スライス優先”ができていないチームほど、品質と速度が安定します。

もし「ここをもう少し深掘りして Claude 向けのプロンプトにしたい」という箇所があれば、
章 21 の中のその部分だけをさらに分解したテキストも出せます。